

# APP2000 Arbeidskrav 1 .

Gruppe 4

Medlemmer:

Rina-Jeanett Nyberg 284894

Aksel Nilsen 286310

Innleveringsdato: 30. September 2026

# KI-støttet applikasjonsutvikling

## Bruk av KI-verktøy til prototype (HTML/CSS)

Jeg testet flere KI-løsninger med denne prompten:

*«Generer en prototype for landingssiden til prosjektoppgaven i APP2000 for studieåret 2025/2026 (turapp), med kun HTML og CSS, statisk nettside uten backend/database.»*

Verktøyene jeg brukte var Replit, Sikt KI og GitHub Copilot.

I Replit fikk jeg generert en enkel landingsside som kunne kjøres direkte i nettleseren. Det gjorde det lett å se resultatet med en gang, og å teste og endre små ting i HTML og CSS fortløpende. Sikt KI var mest nyttig til tekstinnhold, for eksempel å formulere beskrivelser av turappen, turer, hytter og reservasjon på en litt mer ryddig måte. GitHub Copilot ga løpende forslag til både HTML- og CSS-kode i VS Code, og hjalp meg å få et mer strukturert og «profesjonelt» oppsett på selve koden.

I alle tilfellene endte jeg opp med det samme på et overordnet nivå: en prototype, altså en enkel, statisk HTML/CSS-side uten ekte funksjonalitet, backend eller database.

## Begreper knyttet til bruk av KI

I denne sammenhengen kan man bruke KI på to måter:

1. til å jobbe raskere og enklere når man utvikler, og
  2. til å lage funksjoner i selve webapplikasjonen som bruker KI.
- Under forklarer jeg kort noen sentrale begreper, med utgangspunkt i det.

### Prompt

En prompt er teksten som skrives inn til KI for å få noe tilbake. For eksempel ble det brukt en prompt for å få generert landingssiden til turappen. Hvordan prompten formuleres har mye å si for hvor gode svar du får, både når du ber om kode, og når du bygger funksjoner der brukeren skal «snakke» med KI.

### Token

Et token er en liten bit tekst som modellen jobber med. Modellen har en grense for hvor mange tokens den kan håndtere om gangen. I praksis betyr det at det er en grense for hvor mye tekst/kode du kan sende inn og fortsatt få fornuftige svar, både i utvikling og i en ferdig app.

### Copilot

En Copilot er en KI som sitter «ved siden av» deg mens du koder og foreslår kode, slik GitHub Copilot gjør i VS Code. Det gjør at du slipper å skrive alt fra scratch, og du får ofte bedre struktur på koden. Samme idé kan også brukes for sluttbrukere, for eksempel en «Copilot» som hjelper dem å skrive tekst eller planlegge en tur.

## Coding Agent

En coding agent er en litt «smartere» KI som ikke bare foreslår én og én linje, men som kan planlegge og lage større deler av et system ut fra en beskrivelse eller spesifikasjon. Den kan for eksempel generere flere filer, sette opp struktur og lage funksjoner. Den samme typen agent kan også brukes inne i en app, for eksempel til å generere turforslag eller ruteplaner for brukeren.

## Agent Skill

En agent skill er en bestemt ting agenten kan gjøre, for eksempel «lese filer», «kalle et API», «lage HTML», eller «validere et skjema». I utvikling kan slike skills gjøre KI-verktøy mer nyttige. I en app kan du tenke på det som egne funksjoner, for eksempel «finn hytter som passer denne brukeren» eller «lag pakkeliste til fjellturen».

## AI API

Et AI API er en tjeneste på nett som du kan sende tekst eller data til fra koden din, og få et KI-svar tilbake. Dette er måten utviklere kobler KI inn i egne systemer og apper på. Hvis turappen for eksempel skulle hatt en chatassistent eller et anbefalingssystem, ville det typisk vært bygget på et AI API i bakgrunnen.

## AI SDK

Et AI SDK er et bibliotek (for eksempel for JavaScript) som gjør det enklere å bruke et AI API i koden. I stedet for å bygge alt fra bunnen av, får du ferdige funksjoner for å sende forespørsler og håndtere svar.

## AI CLI

En AI CLI er et kommandolinjeverktøy for å bruke KI direkte fra terminalen. Du kan få generert kode, forklart filer eller få forslag til endringer uten å åpne en egen editor. Dette er mest nyttig for utviklere, og ikke noe sluttbrukeren av en turapp ser.

## Model Context Protocol (MCP)

Model Context Protocol (MCP) er en måte å koble KI-modellen til forskjellige kilder på, som filer, API-er eller databaser, på en strukturert og kontrollert måte. Poenget er at modellen skal kunne forstå prosjektet eller appen bedre, uten at man roter med sikkerhet eller struktur. Dette er særlig viktig hvis man skal bygge mer avanserte Kifunksjoner som trenger tilgang til reelle data fra appen.

# Spesifikasjonsdrevet utvikling (SDD)

**Forklaring på SSD og Vibe Coding:** Vibe Coding betyr at man begynner å kode direkte ved hjelp av KI og gjør endringer underveis. Dette kan være raskt, men det kan også gjøre prosjektet mer uoversiktlig etter hvert. Spec-Driven Development (SDD) handler om å lage en tydelig spesifikasjon eller prompt før man begynner å kode. Man starter fra begynnelsen og beskriver steg for steg hva systemet skal gjøre og hvilke krav som gjelder. Da får KI et bedre grunnlag å jobbe ut fra, og den trenger ikke å gjette seg frem til hva du ønsker. Dette gir ofte mer nøyaktig kode, færre misforståelser og en mer strukturert utviklingsprosess. For større prosjekter gjør SDD det også enklere å holde oversikt, siden alle jobber ut fra den samme planen eller "oppskriften".

## Prosjektets mål

Utvikle en moderne, KI-støttet turplattform for TiU som gir brukere tilgang til turruter, hytter og turmål, tilbyr kart og navigering, støtter fellesturer (fast og fleksibel dato), har chat og sosial funksjonalitet, egne portaler for hytteeiere og annonsører, admin-panel, flerspråklighet og integrasjon med vær-API.

## Kjerneprinsipper

- Spesifikasjoner styrer utviklingen (én sannhetskilde).
- Tydelig intensjon og akseptkriterier før implementering.
- Ingen uklarheter.
- Modulbasert system.
- KI-drevet arbeidsflyt: Plan → Implementering → Validering.
- Brukersentrert design.
- Skalerbar, tilgjengelig og sikker løsning.

## Roller

- Vanlig bruker
- Turleder
- Hytteeier
- Annonsør
- Admin

## Systemmoduler

1. Kart
2. Turruter
3. Hytter

4. 4. Fellesturer (fast dato)
5. 5. Fellesturer (fleksibel dato)
6. 6. Chat
7. 7. Brukere og roller
8. 8. Favoritter og anmeldelser
9. 9. GPS-navigering
10. 10. Annonseportal
11. 11. Admin-panel
12. 12. Vær-API
13. 13. Flerspråklighet
14. 14. Database + REST API

## Funksjonelle krav Kart

- Vise ruter, hytter og turmål
- Filtrering
- Tegning av ruter
- GPX-import
- Høydedata
- GPS-navigering

## Turruter

- Registrering via tegning, GPX eller eksisterende stier
- Metadata
- Kobling til hytter'

## Hytter

- Betjent/ubetjent
- Fasiliteter, pris
- Undersider
- Kartplassering

## Fellesturer – fast dato

- Påmelding
- Ledige plasser

- Kartvisning
- Gruppechat

## Fellesturer – fleksibel dato

- Tidsintervall
- Interessepåmelding
- Hytter melder kapasitet
- Vær-API påvirker dato
- Bindende påmelding
- Låsing av dato
- Romreservasjoner
- Rabatt for tidlig påmelding

## Chat

- Direktemeldinger
- Gruppechat
- Bildedeling

## Brukere og roller

- Registrering
- Roller
- Profil
- Favoritter og anmeldelser

## Annonseportal

- Annonserregistrering
- Godkjenning
- Annonser med bilder/tekst
- Kategorier
- Betaling per visning/klikk
- Statistikk

## Admin

- CRUD for alle data
- Godkjenning av turer og annonser

## Vær-API

- Vær per turmål og dato

## Flerspråklighet

- react-i18next
- Språkfiler

## Ikke-funksjonelle krav

- Mobilvennlig
- Responsivt kart
- Sikker autentisering
- Skalerbar backend
- Robust database
- God ytelse på mobil
- Tilgjengelighet (WCAG)

## Systembegrensninger

- React frontend
- REST API backend
- react-i18next
- OpenLayers/Leaflet
- Yr-API
- Database
- Ingen mobilapp-rammeverk

## Akseptansekriterier (kortversjon)

- Søk og filtrering i kart fungerer
- Påmelding til turer fungerer
- Fleksibel dato låses når nok deltakere
- Hytteeiere kan melde kapasitet
- Annonserer kan opprette annonser og se statistikk
- Chat fungerer med bilder
- GPS-navigering fungerer

- Løsningen er flerspråklig

## Edge Cases

- Flere interessepåmeldinger
- Overlappende bindende påmeldinger
- Hytte trekker kapasitet
- Manglende værdata
- Korrupt GPX
- Ugyldig annonsetid
- Mange bilder i chat

## Avklaringer som trengs

- Minimum deltakere for låsing
- Regler for rabatt
- Maks bilder i chat
- Språkvalg
- Betalingsmetoder

## Arkitektur

- React
- Node/Express eller .NET
- PostgreSQL/MongoDB
- REST API
- OpenLayers/Leaflet
- Yr-API
- JWT
- WebSockets

## Oppgave-nedbrytning

**Backend:** API, roller, turer, hytter, annonser, fleksible turer, vær, chat, statistikk

**Frontend:** Kart, filtrering, tur-UI, chat, admin, språk

**Database:** Skjemaer, relasjoner, indekser

## Validering

- Enhetstester
- Integrasjonstester
- Ui-tester
- Belastningstester
- API-tester
- Akseptansetester

KI brukte Microsoft-bloggen og introduksjonsvideoen til å lage en strukturert, profesjonell spesifikasjon for hele turappen.

## Nettsidedesign med CSS Grid

Forklaring :

Vi har laget nettsiden med CSS Grid. Layouten er delt inn i header, navbar, sidebar, innhold og footer. Dette gjør at siden blir enkel å navigere i og gir en ryddig oversikt over innholdet. Sidebaren brukes til filtre, mens hovedområdet viser kart og turinformasjon. CSS Grid gjør det enkelt å organisere innholdet på siden.

## Oppdatere DOM-en

Forklaring:

Vi har laget en HTML-side med en tekstboks og en knapp som er koblet til JavaScript. Når brukeren skriver inn tekst og klikker på knappen, blir teksten lagt til på nettsiden som et nytt element. JavaScript henter teksten fra tekstboksen og legger den inn i listen. Vi har også laget en funksjon som gjør at elementet blir fjernet igjen hvis brukeren klikker på det. På den måten blir innholdet på siden oppdatert med en gang, uten at siden må lastes inn på nytt.

DOM er måten JavaScript kan lese og endre innholdet på nettsiden. Når brukeren legger til eller fjerner elementer, oppdateres siden direkte uten at den må lastes inn på nytt.

## Inndatavalidering i turappen

Forklaring :

Vi har brukt HTML-validering og JavaScript-validering i skjemaet. HTML sjekker at feltene er fylt ut, mens JavaScript gjør ekstra kontroll. Dette gir brukeren rask beskjed hvis noe er feil. I tillegg må data valideres på serveren, fordi validering i nettleseren alene ikke er nok for å sikre at riktig informasjon blir lagret.

## Visning av JSON hyttedata med TypeScript

Forklaring:

Vi har brukt JSON til å lagre data på en strukturert måte. Dataene inneholder et array med varer, der hver vare er et objekt. Vi har også brukt underobjekter for kategori og lagerinformasjon. Dette gjør dataene mer oversiktlige og enklere å jobbe med i TypeScript. JSON-formatet passer godt fordi det er lett å lese og ofte brukes til å utveksle data mellom systemer.